



中科院计算所
INSTITUTE OF COMPUTING TECHNOLOGY, CAS

基于LoongArch架构的ART执行引擎 适配与优化技术研究

鄢玺

导师：张福新

2026年6月

答辩提纲

- 01 研究背景与目标
- 02 执行引擎适配与验证
- 03 性能优化研究与结果
- 04 总结与展望

一、研究背景与目标

问题定义、研究主线与论文贡献



问题定义与主要贡献

核心问题

- AOSP 15 官方主线尚未提供 LoongArch 平台 ART 执行引擎实现。
- 论文必须同时回答“能运行、能验证、能优化”三个问题。
- 性能结论需要统一 baseline 与可复核证据链支撑。

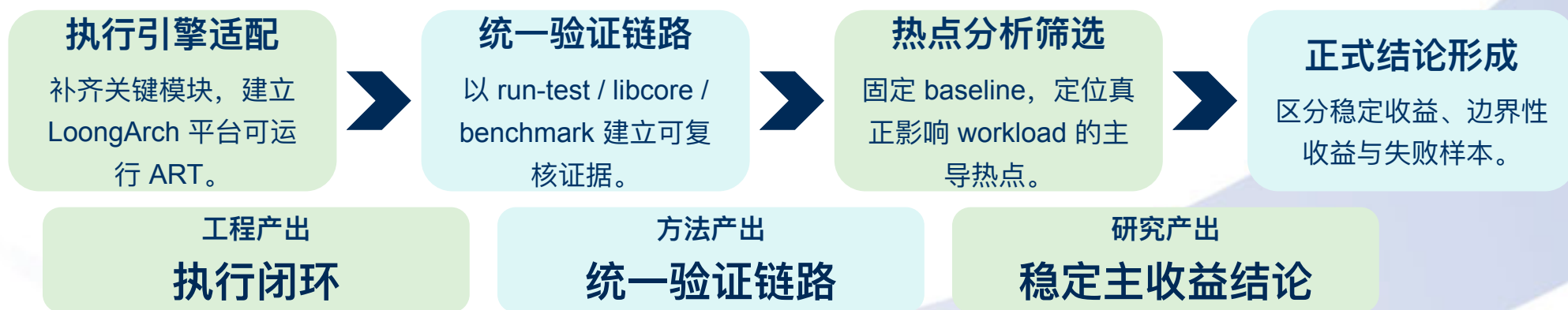
论文回答

- 完成执行引擎关键模块适配，建立 LoongArch 平台 ART 执行闭环。
- 建立编译、部署、run-test、libcore、benchmark 的统一验证链路。
- 筛选三项正式方案，并区分稳定收益、边界性收益与失败样本。

主线不是零散 patch，而是“适配设计 → 验证收敛 → 优化筛选”的系统闭环。

整体研究路线

- 本文从“可运行”推进到“可验证”再到“可优化”，最终给出稳定主结论与收益边界。
- 执行引擎适配解决平台可用性问题。
- 统一验证链路保证语义正确与性能比较口径一致。
- 正式单次对比与重复测量共同约束优化结论的强度。



二、执行引擎适配与验证

建立可运行、可验证的 LoongArch 平台 ART



执行引擎适配的系统闭环

核心目标：以执行路径为主线补齐关键模块，形成可运行、可验证的 LoongArch 平台 ART 执行闭环

从汇编器、解释器到 JIT、JNI 与运行时入口，关键不是单点可跑，而是整条执行链条同时闭合。

Assembler 与基础生成

寄存器、指令编码、分支与链接修补能力。

C++ 解释器与 Nterp

补齐解释执行与字节码派发的关键路径。

JIT 后端与 OSR

打通 HIR 到 LoongArch64 机器码生成。

JNI 编译器与桥接

闭合 Java / native 调用链与栈帧约定。

Runtime Fast Path

补齐字符串、调用桩、deopt 等运行时入口。

Intrinsic 与能力扩展

补充标准库热点路径与执行引擎成熟度。

结论：适配结果是执行引擎系统闭环，不是若干体系结构相关文件的零散移植。



统一验证链路与正确性结果

先证明语义正确，再讨论 benchmark 收益；当前口径以系统级目标机回归为主

编译校验

确认当前源码状态下
ART 关键模块可完整构建。

部署同步

统一推送同一套产物，
保证设备侧与源码状态一致。

目标机回归

以 run-test 做系统级语义验证，再决定是否进入性能评测。

benchmark 评测

仅在前序验证通过后采集分数、日志与 profile。

run-test 用例
1033

compiler 变体
7 类

自动 skip
430

实际执行
6801

run-test 统计口径

按 7 类 compiler 变体展开系统级回归，不与 C++ 解释器 / Nterp / JIT 三条主路径一一对应。

libcore 摘要

设备侧主要测试包级结果通过；少量测试因架构条件限制自动跳过，不影响主要正确性结论。

结论：当前版本已完成 1033 个 run-test 在 7 类 compiler 变体下的系统级目标机回归，并通过 libcore 包级验证，具备进入性能研究阶段的语义基础。

三、性能优化研究

统一 baseline、热点驱动与收益边界

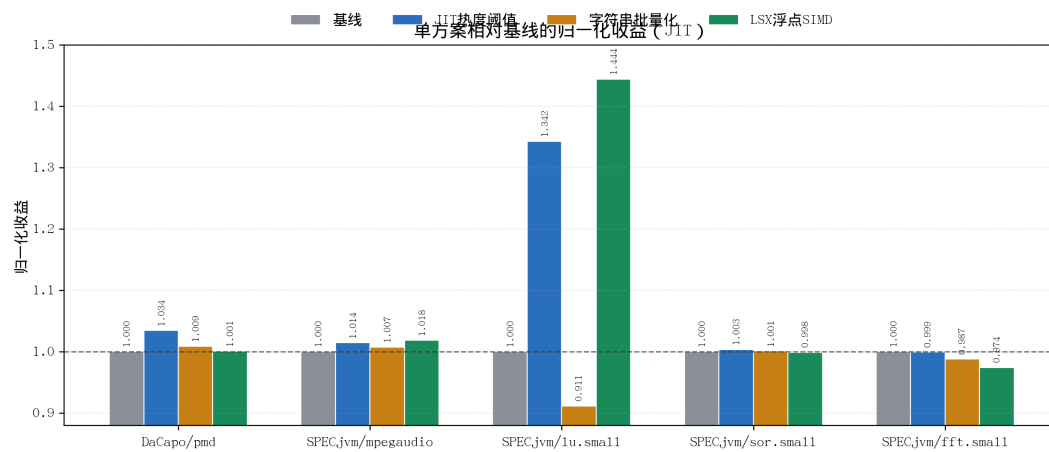
三项正式方案的单次结果总览

统一 **baseline** 下，同日三项正式方案的收益层级已经足够清晰。

方案一：JIT 热度阈值调整，pmd +3.30%，但复测稳定性不足。

方案二：字符串搬运批量化，pmd +0.86%，同时 lu.small -8.89%。

方案三：LSX 浮点 SIMD，lu.small +44.40%，且证据链最完整。



稳定主结论：LSX 浮点 SIMD 向量化

LSX 浮点 SIMD 是当前唯一同时满足收益、解释性与复现性的正式方案。

补齐 float32/float64 的核心 lowering，直接命中 lu.small 主热点。

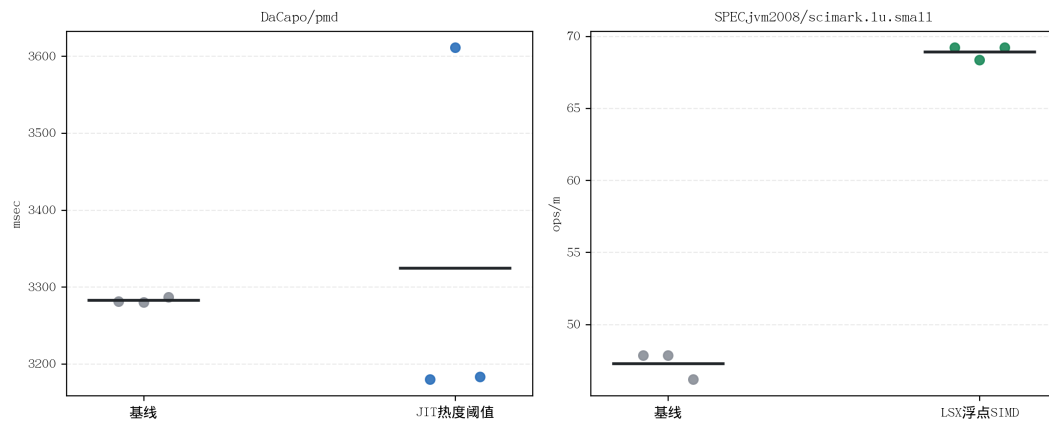
正式单次结果：46.22 → 66.74 ops/m，增益 +44.40%。

同日 3 轮复测均值 +45.70%，标准差仅 0.50 ops/m。

单次增益
+44.40%

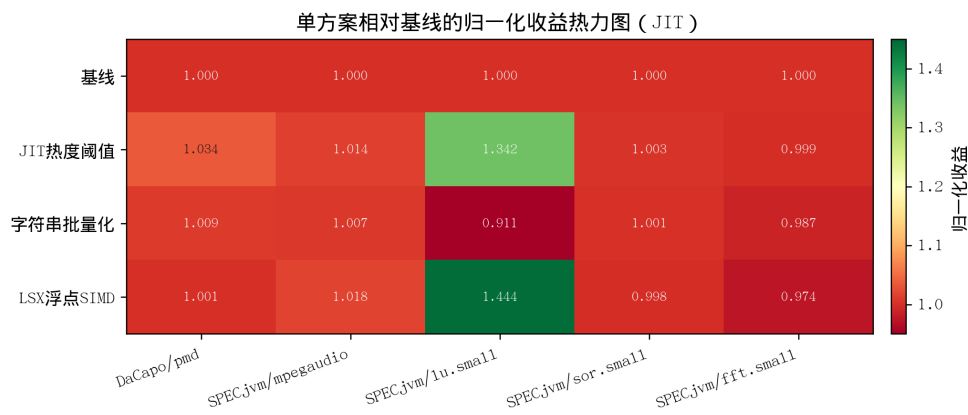
复测均值
+45.70%

同日重复测量：原始散点与均值



收益边界与方法论结论

- 优化是否值得保留，取决于收益边界是否清晰、证据链是否完整。
- 方案一在 pmd 上存在正收益样本，但复测均值尚未稳定转正。
- 方案二说明“局部热点下降”不等于“全局 benchmark 提升”。
- 方案三收益集中在规则浮点循环，并不会自动推广到所有数值 workload。
- 因此，LoongArch 平台 ART 优化应优先命中主导热点，并严格控制副作用。



四、总结与展望

论文结论、方法价值与后续工作

总结与展望

论文主结论

- AOSP 15 ART 在 LoongArch 平台上的执行引擎适配是可行的。
- 统一验证链路与固定 baseline 是性能研究具备解释价值的前提。
- 只有命中主导热点、边界清晰且可复现的方案才值得保留。

后续工作

- 完善 AOT / 安装期编译支持。
- 扩展 LSX / LASX 能力与更高层门控。
- 引入更多 benchmark 与应用级验证。
- 增强低扰动观测与运行时因果分析。

本文的核心产出不仅是工程闭环，更是“什么值得保留、为什么成立、边界在哪里”的研究方法。

感谢聆听！

请各位老师批评指正

Q&A